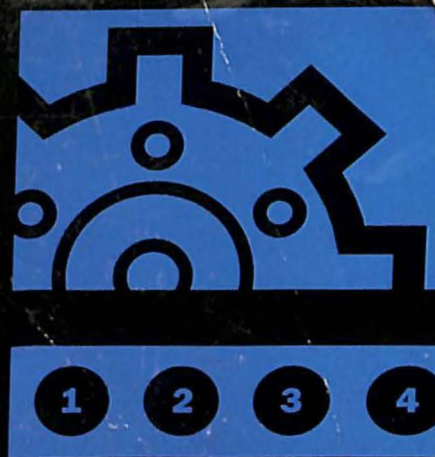
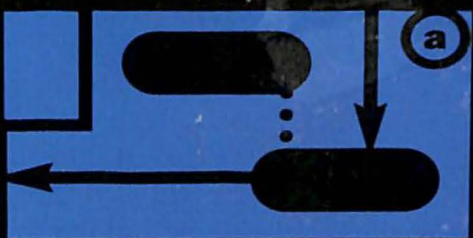
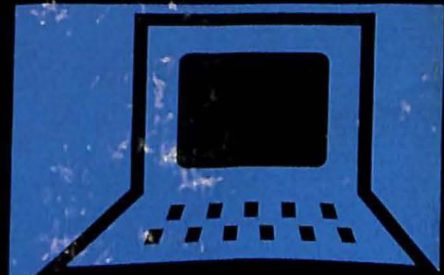
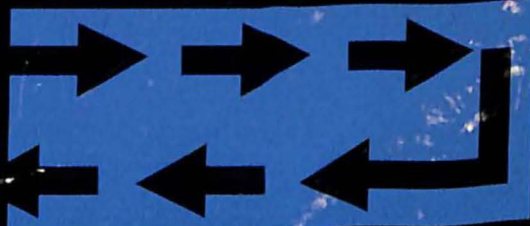


$$\Gamma_L(f) = \frac{Z_L(f)}{Z(f)}$$



Programación en Linux

C O N E J E M P L O S



```
printf("\n\n\n\n\nWhat is the  
name? "); gets(item.name);  
printf("What is the city?  
"); gets(item.city);  
getzip(item.zipcode); ans = g  
char(); if((fp=fopen(FILENAME,  
"r+")) == NULL){ err_msg("R  
Read error-ensure file retu
```



Prentice
Hall

que®

Kurt Wall

KURT WALL

Programación en Linux

C O N E J E M P L O S



**BIBLIOTECA
UNIVERSIDAD de PALERMO**

Prohibida su Reproducción - Ley 11723



**Argentina • Bolivia • Brasil • Colombia • Costa Rica • Chile • Ecuador • Salvador •
España • Guatemala • Honduras • México • Nicaragua • Panamá • Paraguay • Perú •
Puerto Rico • República Dominicana • Uruguay • Venezuela**

**Amsterdam • Harlow • Londres • Menlo Park • Miami • Munich • Nueva Delhi • Nueva Jersey •
Nueva York • Ontario • París • Singapur • Sydney • Tokio • Toronto • Zurich**

datos de catalogación bibliográfica

519.68
WAL

Wall, Kurt

Programación en Linux con ejemplos - 1ª ed -

Buenos Aires: Prentice Hall, 2000

568 p.; 24x19 cm

Traducción de: Jorge Gorín

ISBN: 987-9460-09-X

I. Título - 1. Programación

UNIVERSIDAD DE PALERMO
BIBLIOTECA

042613 ✓

Fecha recepción: 08.10.03

Procedencia: COMPRA

Editora: María Fernanda Castillo
Armado de interior y tapa: Pérez Villamil & Asociados
Traducción: Jorge Gorín
Producción: Marcela Mangarelli

Traducido de:
Linux Programming by example, by Kurt Wall, published by QUE.
Copyright © 2000,
All Rights Reserved.
Published by arrangement with the original publisher,
PRENTICE HALL, inc, A Pearson Education Company.
ISBN: 0-7897-2215-1

Edición en Español publicada por Pearson Education, S.A.
Copyright © 2000
ISBN: 987-9460-09-X

Este libro no puede ser reproducido total ni parcialmente en ninguna forma, ni por ningún medio o procedimiento, sea reprográfico, fotocopia, mimeografía, mimeográfico o cualquier otro sistema mecánico, fotoquímico, electrónico, informático, magnético, electroóptico, etcétera. Cualquier reproducción sin el permiso previo por escrito de la editorial viola derechos reservados, es ilegal y constituye un delito.

© 2000, PEARSON EDUCATION S.A.
Av. Regimiento de Patricios 1959 (1266), Buenos Aires,
República Argentina

Queda hecho el depósito que dispone la ley 11.723
Impreso en Perú. Printed in Perú.
Impreso en Quebecor Perú, en el mes de noviembre de 2000
Primera edición: Diciembre de 2000

Associate Publisher
Dean Miller

Executive Editor
Jeff Koch

Acquisitions Editor
Gretchen Ganser

Development Editor
Sean Dixon

Managing Editor
Lisa Wilson

Project Editor
Tonya Simpsons

Copy Editor
Kezia Endsley

Indexer
Cheryl Landes

Proofreader
Benjamin Berg

Technical Editor
Cameron Laird

Team Coordinator
Cindy Teeters

Interior Design
Karen Ruggles

Cover Design
Duane Rader

Copy Writer
Eric Borgert

Production
Dan Harris
Heather Moseman

Semáforos y colas de mensajes

Este capítulo continúa con el tratamiento del IPC System V que fue comenzado en el capítulo anterior. El análisis del mismo será completado con la cobertura de las colas de mensajes y los semáforos.

Este capítulo cubre los siguientes temas:

- Qué es una cola de mensajes
- Creación de una cola de mensajes
- Agregado de elementos a una cola de mensajes existente
- Publicación y recuperación de mensajes
- Eliminación de una cola de mensajes
- Qué son los semáforos
- Creación y eliminación de semáforos
- Actualización de semáforos

✓ La utilización de objetos IPC de memoria compartida se analiza en el capítulo 16, "Memoria compartida".

Todos los programas de este capítulo pueden ser encontrados en el sitio Web <http://www.mcp.com/info> bajo el número de ISBN 0789722151.

El IPC System V y Linux

El método de comunicación entre procesos IPC System V es sumamente conocido y habitualmente empleado, pero la implementación del mismo por parte de Linux tiene numerosas imprecisiones, como se hizo notar en el capítulo anterior y se lo continuará haciendo en éste. La versión de Linux del IPC System V también es anterior al IPC POSIX, pero son pocos los programas de Linux que en la práctica la implementan, aun cuando se encuentre disponible en los kernels correspondientes a la versión 2.2.x. El IPC POSIX ofrece una interfaz similar a la del System V comentada en este capítulo y en el anterior, pero elimina por un lado algunos de los problemas que tenía el System V y por el otro simplifica la interfaz. El problema es que aunque el IPC System V es estándar, está implementado en Linux de manera deficiente y casi perversa, por razones que son demasiado avanzadas para ser cubiertas aquí.

El resultado es que Linux, que trata denodadamente de satisfacer las normas POSIX (y en general tiene éxito), implementa una versión demasiado antigua tanto del IPC POSIX como del IPC System V. La dificultad es que el System V se encuentra sumamente difundido y es más común, pero la versión POSIX es mejor, más sencilla de utilizar y cuenta con una interfaz más uniforme para poder interactuar desde un programa con los tres tipos de objetos IPC. ¿El resultado? Personalmente elegí romper con mi propia regla y opté por cubrir lo que es probable que se encuentre tanto en los programas existentes como los nuevos en lugar de explicar el Método Correcto ©; o sea, el IPC POSIX.

Cuando se programa utilizando semáforos surge otra cuestión adicional. Los semáforos de System V fueron creados en la Edad de las Tinieblas para poder abordar la multitud de problemas que surgen cuando varios hilos de un único proceso (y también de múltiples procesos) que se encuentran en ejecución necesitan acceder a los mismos recursos de sistema aproximadamente al mismo tiempo. Si bien considero que los programas multi-hilos bien *escritos* son un componente esencial (en realidad, indispensable) de cualquier sistema Linux, lo que deseo enfatizar en realidad es bien escritos. La redacción de programas multi-hilos se encuentra lejos, muy lejos de los alcances de este libro; no es una tarea que deje lugar para programadores novicios. Los programadores experimentados, hasta los súper-programadores, tratarán de encontrar una solución alternativa antes de recurrir a la programación multi-hilos porque la misma es difícil de llevar a cabo correctamente.

¿Mi solución? He procedido a simplificar la discusión de los semáforos porque la versión System V fue creada pensando en los procesos multi-hilos. No obstante, los semáforos empleados en programas estándar, mono-hilo, son muy útiles, como lo apreciará el lector en este capítulo. La interfaz POSIX de semáforos es más sencilla pero su empleo no se encuentra muy difundido, por el momento, en los programas de Linux.

Colas de mensajes

Una cola de mensajes es una lista vinculada de mensajes almacenada dentro del kernel e identificada a los procesos de usuario por un *identificador de cola de mensajes*, un identificador del tipo analizado en el capítulo anterior.

Por razones de brevedad y conveniencia, este capítulo utiliza los términos cola e identificador de cola para referirse a las colas de mensajes y a los identificadores de colas de mensajes. Si el lector añade un mensaje a una cola, la misma aparentará ser un FIFO porque los nuevos mensajes son agregados al final de la cola de mensajes. Sin embargo, el lector no tiene que recuperar los mensajes en orden de arribo como en un FIFO. Las colas de mensajes pueden ser consideradas una forma simple de memoria asociativa porque, tal como se verá más adelante, uno puede utilizar el tipo de un mensaje para recuperar el mensaje fuera de secuencia. La diferencia entre las colas de mensajes y los FIFOs se ilustra en la figura 17-1.

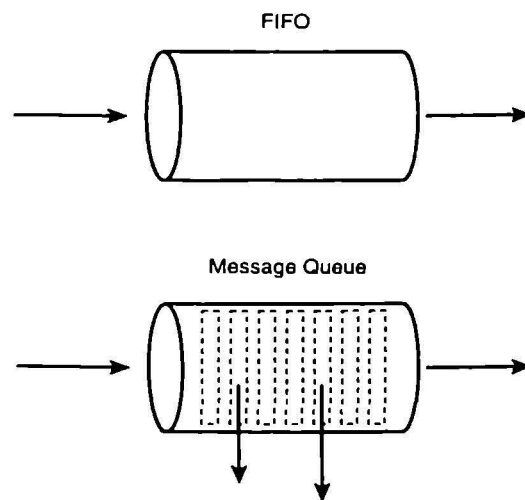


Figura 17.1. A diferencia de lo que ocurre con las FIFOs, los mensajes pueden ser leídos desde una cola de mensajes en cualquier orden deseado.

Como el lector ya sabe, desde un FIFO los datos son leídos en el mismo orden en que fueron escritos al mismo. Esto está ilustrado en la mitad superior de la figura 17-1. Sin embargo, si uno conoce el tipo de un mensaje, puede extraerlo de la cola fuera de secuencia. Como lo indican las flechas que salen y se alejan de la cola de mensajes, los elementos integrantes de las mismas son en general leídos en orden FIFO, o sea *el primero que se escribe es el primero que se lee*. Los dos rectángulos dibujados con líneas de puntos con flechas apuntando hacia afuera de la cola de mensajes muestran cómo pueden ser leídos los mensajes en orden arbitrario.

Todas las funciones de manipulación de colas de mensajes se encuentran declaradas en `<sys/msg.h>`, pero se debe también incluir en el código fuente del programa `<sys/types.h>` y `<sys/ipc.h>` para acceder al tipo de variables y constantes que contienen sus declaraciones. Para crear una nueva cola o para abrir una cola existente se deberá emplear la función `msgget`. Para añadir un nuevo mensaje al final de una cola, utilice `msgsnd`. Para obtener un mensaje de la cola, emplee `msgrcv`. La llamada a `msgctl` le permite a uno tanto manipular las prestaciones de la cola como eliminar la misma, siempre y cuando el proceso que efectúe la llamada sea el creador de la cola o cuente con permisos de superusuario.

Creación y apertura de una cola

La función `msgget` crea una cola nueva o abre una ya existente. Su prototipo es el siguiente:

```
int msgget(key_t key, int flags);
```

Si la llamada resulta exitosa, retorna el identificador de la cola nueva o existente que corresponda al valor contenido en `key` (*clave*), de acuerdo con lo siguiente:

- Si `key` es `IPC_PRIVATE`, se crea una nueva cola empleando un valor de clave que genera la implementación del sistema operativo de que se trate. Utilizando `IPC_PRIVATE` se garantiza que se cree una nueva cola siempre y cuando no sean excedidos con la misma el número total de colas o el número total de bytes disponibles para la totalidad de las colas que permita el sistema operativo.
- Si `key` no es `IPC_PRIVATE`, pero `key` no corresponde a una cola existente que tenga idéntica clave, o si se encuentra activado asimismo el bit `IPC_CREAT` en el argumento `flags`, la cola será igualmente creada.
- En el caso restante —o sea, si `key` no es `IPC_PRIVATE` y el bit de `IPC_CREAT` de `flags` no se encuentra activado— `msgget` retorna el identificador de la cola existente asociada con `key`.

Si `msgget` fracasa, retorna -1 y asigna a la variable de error `errno` el valor adecuado.



EJEMPLO

Ejemplo

El programa de demostración que sigue, `crear_cola`, crea una nueva cola de mensajes. Si se lo vuelve a ejecutar una segunda vez, en lugar de crear la cola especificada simplemente abre la cola existente.

```
/* Nombre del programa en Internet: mkq.c */
/*
 * crear_cola.c - Crea una cola de mensajes de IPC System V
 */
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[])
{
    int identificador_cola;
    key_t clave = 123;    /* Clave de la cola */

    /* Crear la cola de mensajes */
    if((identificador_cola == msgget(clave, IPC_CREAT | 0666)) < 0) {
```

```

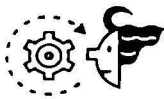
        perror("msgget:create");
        exit(EXIT_FAILURE);
    }
    printf("Creada cola de identificador = %d\n", identificadorCola);

    /* Open the queue again */
    if((identificadorCola == msgget(key, 0)) < 0) {
        perror("msgget:open");
        exit(EXIT_FAILURE);
    }
    printf("Abierta cola de identificador = %d\n", identificadorCola);

    exit(EXIT_SUCCESS);
}

```

La salida de este programa en mi sistema presentó el siguiente aspecto:



SALIDA

```

$ ./crearCola
Creada cola de identificador = 384
Abierta cola de identificador = 384

```

Si la primera llamada a `msgget` tiene éxito, `crearCola` exhibe el identificador de la cola recién creada y luego llama a `msgget` una segunda vez. Si la segunda llamada también tiene éxito, `crearCola` informa esto de nuevo, pero la segunda llamada meramente abre la cola existente en lugar de crear una cola nueva. Obsérvese que la primera cola especifica permisos de lectura/escritura para todos los usuarios empleando la notación octal estándar.

PRECAUCIÓN

A diferencia de lo que ocurre con el comportamiento de la función `open`, cuando se crea una estructura de IPC System V, la `umask` del proceso no modifica los permisos de acceso a la estructura. Si uno no establece permisos de acceso, el modo predeterminado es `0`, lo que significa que ni el creador de la estructura tendrá permisos de acceso de lectura/escritura a la misma!

Escritura de un mensaje a una cola

Tal como fue explicado anteriormente, para añadir un nuevo mensaje al final de una cola se deberá utilizar la función `msgsnd`, cuyo prototipo es el siguiente:

```
int msgsnd(int msqid, const void *ptr, size_t nbytes, int flags);
```

`msgsnd` retorna `0` si tiene éxito, y en caso de fracasar retorna `-1` y asigna a la variable de error `errno` uno de los siguientes valores:

- `EAGAIN`
- `EACCES`

- EFAULT
- EIDRM
- EINTR
- EINVAL
- ENOMEM

✓ Los valores posibles de `errno` y las explicaciones de los mismos están listados en la tabla 6.1, “Códigos de error generados por las llamadas a sistema”, página 125.

El argumento `msqid` (*identificador de cola de mensajes*) debe ser un identificador de cola retornado por una llamada anterior a `msgget`.

Por su parte, `nbytes` es el número de bytes de la cola publicada, que no tiene que estar terminada en un cero binario.

El argumento `ptr` es un puntero a una estructura `msgbuf`, la cual consiste de un tipo de mensaje y de los bytes de datos que comprenden el mismo.

La estructura `msgbuf` (*buffer de mensaje*) se encuentra definida en `<sys/msg.h>` de la siguiente manera:

```
struct msgbuf {
    long mtype;
    char mtext[1];
};
```

Esta declaración de estructura es en realidad sólo una plantilla, ya que `mtext` debe ser del tamaño de los datos que están siendo almacenados, el cual corresponde al valor de longitud de cadena transferido en el argumento `nbytes`, menos cualquier posible cero de terminación. `mtype` puede ser cualquier entero de tipo `long` mayor que cero. El proceso que efectúa la llamada debe contar también con permiso de acceso a la cola para escritura.

El argumento `flags` (*indicadores*), finalmente, puede ser `0` o `IPC_NOWAIT`. Este último valor ocasiona un comportamiento similar al del indicador `O_NONBLOCK` que se transfiere a la llamada a sistema `open`: si ya sea el número total de mensajes individuales que forman la cola o el tamaño de la misma, en bytes, es igual al límite especificado por el sistema para el mismo, `msgsnd` retorna inmediatamente y asigna a `errno` el valor `EAGAIN`. Como resultado de ello, uno no podrá añadir más mensajes a la cola hasta que por lo menos uno de los mensajes haya sido leído.

Si `flags` es `0` y ya sea que la cola tenga el máximo de mensajes permitidos o haya sido escrito a la cola el número total de bytes de datos permitidos, la llamada a `msgsnd` se bloquea (no retorna) hasta que dicha condición sea modificada. Para modificar esa condición se debe o bien leer mensajes de la cola, eliminar la misma (lo cual asigna a `errno` el valor `EIDRM`) o aguardar a que sea interceptada una señal y el handler correspondiente retorne (lo que asigna a `errno` el valor `EINTR`).

CONSEJO

La plantilla de la estructura `msgbuf` puede ser expandida de modo de satisfacer las necesidades de las distintas aplicaciones. Por ejemplo, si uno desea transferir un mensaje que consista de un valor entero y un arreglo de tipo carácter compuesto por 10 bytes, sólo hace falta declarar `msgbuf` como sigue:

```

struct buffer_mensaje{
    long tipo_mensaje;
    int i;
    char texto_mensaje[10];
};

```

msgbuf es simplemente un valor long seguido por los datos del mensaje, que pueden estar formateados como uno lo considere adecuado. El tamaño de la estructura declarada en este ejemplo es `sizeof(msgbuf) - sizeof(long)`.



EJEMPLO

Ejemplo

Este programa, `enviar_aCola`, añade un mensaje al final de una cola que ya existe. El identificador de la cola le debe ser transferido al programa como único argumento de su línea de comandos.

```

/* Nombre del programa en Internet. qsnd.c */
/*
 * enviar_aCola.c - Enviar un mensaje a una cola ya abierta con anterioridad
 * Sintaxis: enviar_aCola identificador de cola
 */
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define TAMAÑO_BUF 512

struct mensaje {                                /* Estructura para el mensaje */
    long tipo_mensaje;
    char texto_mensaje[TAMAÑO_BUF];
};

int main(int argc, char *argv[])
{
    int identificadorCola;
    int tamañoTexto;                            /* Longitud del mensaje a
ser enviado */
    struct mensaje buffer_mensaje;              /* Estructura de patron mensaje */
    /* Obtener el identificador de cola transferido en la línea de comandos */

```

```

        if(argc != 2) {          /* A saber: nombre del programa y argumento de su línea
de comandos */
            puts("MODULO DE EMPLEO: enviar_aCola <identificador de cola>");
            exit(EXIT_FAILURE);
        }
        identificadorCola = atoi(argv[1]);

        /* Obtener el mensaje que será agregado a la cola */
        puts("Ingrese un mensaje para publicar:");
        if((fgets(&buffer_mensaje->texto_mensaje, TAMANO_BUF, stdin)) == NULL) {
            puts("No hay mensaje para ser publicado");
            exit(EXIT_SUCCESS);
        }

        /* Asociar el mensaje ingresado con este proceso */
        buffer_mensaje.tipo_mensaje = getpid();
        /* Añadir el mensaje al final de la cola */
        tamañoTexto = strlen(buffer_mensaje.texto_mensaje);
        if((msgsnd(identificadorCola, &buffer_mensaje, tamañoTexto, 0)) < 0) {
            perror("msgsnd");
            exit(EXIT_FAILURE);
        }
        puts("Mensaje publicado");

        exit(EXIT_SUCCESS);
    }

```

Una corrida de prueba de este programa produjo la siguiente salida. Obsérvese que el programa utiliza el identificador de cola retornado por la llamada a `crearCola` inmediatamente anterior.



SALIDA

```

$ ./crearCola
Creada cola de identificador = 640
Abierta cola de identificador = 640
$
$ ./enviar_aCola 640
Ingrese un mensaje para publicar:
Este es el mensaje de prueba numero uno
Mensaje publicado

```

El primer programa, `crearCola`, creó una cola nueva cuyo identificador fue 640. El segundo programa, `enviar_aCola 640`, solicitó un mensaje para ser publicado y almacenó la respuesta tipeada (indicada en negritas) directamente en la estructura de patrón mensaje declarada al comienzo del programa.

ma. Si `msgsnd` se completa exitosamente, el programa exhibe un mensaje a tal efecto. Obsérvese que `enviar_aCola` establece el tipo del mensaje al PID del proceso que efectuó la llamada. Esto le permite a uno recuperar más tarde (utilizando `msgrcv`) sólo los mensajes que publicó este proceso.

Obtención de un mensaje presente en una cola de mensajes

Para extraer un mensaje de una cola se debe utilizar `msgrcv` (*recibir mensaje*), que tiene la siguiente sintaxis:

```
int msgrcv(int msqid, void *ptr, size_t nbytes, long type, int flags);
```

Si tiene éxito, `msgrcv` elimina de la cola el mensaje que haya extraído. Los argumentos son los mismos que los que acepta `msgsnd`, excepto que `msgrcv` rellena la estructura señalada por `ptr` con el tipo de mensaje y hasta `nbytes` de datos. El argumento adicional, `type`, corresponde al miembro `tipo_mensaje` de la estructura `mensaje` comentada anteriormente. El valor de `type` determina qué mensaje es retornado, tal como se indica en la siguiente lista:

- Si `type` es 0 se retorna el primer mensaje (el superior) de la cola.
- Si `type` es > 0 se retorna el primer mensaje cuyo `tipo_mensaje` sea igual a `type`.
- Si `type` es < 0 se retorna el primer mensaje cuyo `tipo_mensaje` sea el valor más bajo menor o igual al valor absoluto de `type`.

El valor de `flags` controla además el comportamiento de `msgrcv`. Si el mensaje retornado tiene una extensión mayor a `nbytes` y en `flags` se encuentra activado el bit correspondiente a `MSG_NOERROR`, el mismo resulta truncado a `nbytes` (pero no se genera notificación al respecto). Si el bit respectivo no se encuentra activado, `msgrcv` retorna -1 a fin de indicar un error y asigna a `errno` el valor `E2BIG` (*ERROR: DEMASIADO GRANDE*). El mensaje seguirá permaneciendo en la cola.

Si en `flags` se encontrara activado el bit `IPC_NOWAIT` (*IPC_NOESPERAR*) y no se encontrara disponible un mensaje del tipo especificado, `msgrcv` retorna inmediatamente y asigna a `errno` el valor `ENOMSG`. De lo contrario, `msgrcv` se bloquea (no retorna) hasta que tenga lugar para `msgsnd` una de las mismas condiciones descriptas anteriormente.

NOTA

Se puede emplear un valor negativo para el argumento `tipo` para crear un tipo de cola denominado LIFO o, último que entra primero que sale (last in - first out) a menudo denominado pila (stack). La transferencia como tipo de un valor negativo permite a uno recuperar mensajes de un tipo dado en el orden inverso al que fueron almacenados en la cola.



EJEMPLO

Ejemplo
El siguiente programa, `leerCola`, lee un mensaje de una cola previamente creada que disponga de mensajes. El identificador de la cola de la cual se leerá se le transfiere al programa como un argumento en la línea de comandos del mismo.


```

/* Nombre del programa en Internet: qrd.c */
/*
 * leerCola.c - Read all message from a message queue
 * Sintaxis: leerCola identificador de cola
 */
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <stdio.h>
#include <stdlib.h>

#define TAMAÑO_BUF 512

struct mensaje {
    long tipo_mensaje;
    char texto_mensaje[TAMAÑO_BUF];
};

int main(int argc, char *argv[])
{
    int identificadorCola;
    int tamañoTexto;
    struct mensaje bufferMensaje;

    /* Longitud del mensaje a ser enviado */
    /* Estructura de patron mensaje */

    /* Obtener el identificador de cola transferido en la línea de comandos */
    if(argc != 2) {
        /* A saber: nombre del programa y argumento de su línea de comandos */
        puts("MODULO DE EMPLEO: leerCola <identificador de cola>");
        exit(EXIT_FAILURE);
    }
    identificadorCola = atoi(argv[1]);

    /* Recuperar un mensaje de la cola y exhibirlo */
    tamañoTexto = msgrcv(identificadorCola, & bufferMensaje, TAMAÑO_BUF, 0, 0);
    if(tamañoTexto > 0) {
        printf("Leyendo identificador de cola: %05d\n", identificadorCola);
        printf("\tTipo de mensaje: %05d\n", (&bufferMensaje)->tipo_mensaje);
        printf("\tTexto del mensaje: %s\n", (&bufferMensaje)->texto_mensaje);
    } else {
        perror("msgrcv");
    }
}

```

```

exit(EXIT_FAILURE);
}
exit(EXIT_SUCCESS);
}

```

A continuación se muestra la salida de este programa. En este caso, el mismo utiliza el identificador de cola creado por `crearCola` y lee el mensaje publicado por `enviar_aCola`.



SALIDA

```

$ ./crearCola
Creada cola de identificador = 640
Abierta cola de identificador = 640
$
$ ./enviar_aCola 640
Ingrase un mensaje para publicar:
Este es el mensaje de prueba numero uno
Mensaje publicado
$
$ ./leerCola 640
Leyendo identificador de cola: 00640
    Tipo de mensaje: 14308
    Texto del mensaje: Este es el mensaje de prueba numero uno

```

Se puede advertir, observando el código fuente, que leer de una cola de mensajes es más sencillo que escribir a la misma y requiere de menos código. Resulta de particular interés en el programa de demostración que el mismo obtiene el primer mensaje que encuentra al comienzo de la cola porque al mismo se le transfirió 0 como argumento `type`. En este caso, como el PID del proceso que escribió el mensaje se conoce o puede ser fácilmente obtenido (14308), `leerCola` podría haber transferido 14308 como `type` y haber recuperado de la cola el mismo mensaje.

Manipulación y eliminación de colas de mensajes

La función `msgctl` provee un cierto grado de control sobre las colas de mensajes. Su prototipo es el siguiente:

```
int msgctl(int msqid, int cmd, struct msqid_ds *buf);
```

`msqid`, como de costumbre, es el identificador de una cola existente.

`cmd` (en este caso, acción) puede adoptar uno de los siguientes valores:

- **IPC_RMID:** elimina la cola de estructura `msqid_ds` cuyo identificador es `msqid`.
- **IPC_STAT:** rellena `buf` con el contenido de la cola de estructura `msqid_ds` identificada por `msqid`. `IPC_STAT` le permite a uno desplazarse por los mensajes contenidos en una cola sin eliminar ninguno de ellos. Como `IPC_STAT` lleva a cabo una lectura no destructiva, se la puede considerar similar a `msgrcv`.

- **IPC_SET:** le permite a uno modificar los siguientes parámetros de una cola: UID, GID, modo de acceso y el máximo número de bytes que se permite almacenar en la misma.

**EJEMPLO****Ejemplo**

El siguiente programa utiliza la llamada a `msgctl` para eliminar una cola cuyo identificador se le transfiere en la línea de comandos.

```
/* Nombre del programa en Internet: qctl.c */
/*
 * eliminarCola.c - Elimina una cola de mensajes
 * Sintaxis eliminar mensaje identificador de cola
 */
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/msg.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

int main(int argc, char *argv[])
{

    int identificadorCola;
    struct msqid_ds colaDeMensajes;

    if(argc != 2) {
        puts("MODO DE EMPLEO: eliminarCola <identificadorCola>");
        exit(EXIT_FAILURE);
    }
    identificadorCola = atoi(argv[1]);

    if((msgctl(identificadorCola, IPC_RMID, NULL)) < 0) {
        perror("msgctl");
        exit(EXIT_FAILURE);
    }
    printf("Cola %d eliminada\n", identificadorCola);
    exit(EXIT_SUCCESS);
}
```



```

S ./crearCola
Creada cola de identificador = 1280
Abierta cola de identificador = 1280
S
S ipcs -q
--- Message Queues ---
key          msqid   owner      perms    used-bytes   messages
0x0000007b   1280    kurt_wall  666      0             0
S
S ./eliminarCola 1280
Cola 1280 eliminada
S
S ipcs -q
--- Message Queues ---
key          msqid   owner      perms    used-bytes   messages

```

La salida de este programa muestra que la cola especificada ha sido eliminada. El proceso `crearCola` crea en efecto la cola. El comando `ipcs` confirma que la cola fue creada. Empleando luego el identificador de cola retornado por `crearCola`, `eliminarCola` llama a `msgctl`, especificando el indicador `IPC_RMID` que es requerido por esa rutina para eliminar la cola. Una segunda corrida de `ipcs` confirma que `eliminarCola` efectivamente eliminó la cola de mensajes.

El comando `ipcs(1)` (*estructuras de IPC*), utilizado en varios de los programas de demostración, muestra el número y el estado de todas las estructuras de IPC System V que se encuentran presentes en el sistema cuando en esa instancia de su ejecución.

El comando `ipcrm(1)` eliminará la estructura IPC cuyo tipo e identificador sea especificado en la línea de comandos. Ver las páginas del manual para obtener mayor información. La figura 17-2 muestra la salida del comando `ipcs`.

```

ipcs
----- Shared Memory Segments -----
key      shmid   owner      perms    bytes    nattch   status
0x00000000 395    kwall      666      4096     0
0x7b019831 1522    kwall      600      1024     1      dest
0x00000000 1523    kwall      666      4096     0

----- Semaphore Arrays -----
key      semid   owner      perms    nsems    status
0x00000000 0       kwall      666      1
0x00000000 1       kwall      666      1

----- Message Queues -----
key      msqid   owner      perms    used-bytes   messages
0x0000007b 128     kwall      666      0             0

```

Figura 17.2. El comando `ipcs` muestra todos los objetos del IPC System V que se encuentran presentes en el momento de correr el mismo.

La figura 17-2 muestra que por lo menos uno de cada tipo de objeto IPC existe. Se encuentran corrientemente en uso tres segmentos de memoria compartida, una cola de mensajes y dos semáforos (los semáforos se analizan en el próximo título). Para cada objeto se listan claramente su clave,

identificador, propietario/creador y permisos de acceso. Las estadísticas relativas a cada tipo de objeto se muestran en una o dos columnas del extremo derecho de la tabla, según sea el tipo del objeto.

Semáforos

Los *semáforos* controlan el acceso a los recursos compartidos. Son sumamente diferentes de todas las demás formas de IPC que se han visto hasta ahora, porque no ponen información a disposición de los procesos sino que en cambio sincronizan el acceso a los recursos compartidos que no deben de ser accedidos al mismo tiempo por más de un proceso. A ese respecto, las operaciones con semáforos se parecen más a una generalización del bloqueo de archivos, porque se aplican a más recursos que sólo archivos. Esta parte analiza sólo la modalidad más simple de un semáforo, el semáforo binario. Un *semáforo binario* puede adoptar sólo uno de dos valores: 0 cuando un recurso se encuentra bloqueado y no debe ser accedido por otros procesos, y 1 cuando el recurso queda desbloqueado.

✓ Para obtener más información acerca del bloqueo de archivos y registros, ver “Bloqueo de archivos”, página 182.

Los semáforos funcionan de una manera muy similar a señales de tránsito de sólo dos luces (roja y verde) ubicadas en un cruce transitado. Cuando un proceso necesita acceder a un recurso controlado, tal como un archivo, primero verifica el valor del semáforo pertinente, lo mismo que un conductor verifica que una luz de tránsito esté en verde. Si el semáforo tiene el valor 0, que es equivalente binario de la luz roja, el recurso se halla en uso, de modo que el proceso se bloquea hasta que el recurso deseado se vuelva disponible (es decir, el valor del semáforo se vuelva no cero. En la terminología empleada por el IPC System V, este bloqueo temporario se denomina *wait* (*espera*). Si el semáforo tiene un valor positivo, lo cual equivale a una luz verde para dicho acceso, el recurso asociado al mismo se encuentra disponible, de modo que el semáforo procede a disminuir el semáforo (enciende la luz roja), lleva a cabo sus operaciones con el recurso y luego vuelve a encender la luz verde, es decir, incrementa el valor del semáforo a fin de liberar su “bloqueo”.

Creación de un semáforo

Naturalmente, antes de que un proceso esté en condiciones de incrementar o disminuir un semáforo, y suponiendo que el proceso cuente con los permisos adecuados, el semáforo debe existir. La función para crear un nuevo semáforo o acceder a uno existente es la misma, `semget`, prototipada en

✓ `<sys/sem.h>` de la siguiente manera (se debe también incluir en el código fuente los archivos de encabezado `<sys/ipc.h>` y `<sys/types.h>`):

```
int semget(key_t key, int nsems, int flags);
```

`semget` retorna el identificador del semáforo asociado con un conjunto de semáforos cuyo número es `nsems`. Si `key` (*clave*) es `IPC_PRIVATE` o si `key` no se encuentra ya en uso y además en `flags` se encuentra activado el bit `IPC_CREAT`, el semáforo será creado. Lo mismo que en el caso de los segmentos de memoria compartida y las colas de mensajes, para establecer los modos de

acceso al semáforo `flags` puede ser también objeto de una operación lógica de 0 bit a bit con los bits de permiso, expresados en notación octal. Obsérvese, sin embargo, que los semáforos deben contar con permisos de lectura y de *modificación* en lugar de con permisos de lectura y *escritura*. Los semáforos emplean el concepto de *modificación* en lugar del de *escritura* porque nunca en realidad se escriben datos a un semáforo, simplemente se altera (o modifica) su estado incrementando o disminuyendo su valor. Si ocurre algún error `semget` retorna -1 y asigna a `errno` un valor adecuado. Si todo anduvo bien, retorna al proceso que la llamó el identificador del semáforo asociado con el valor de `key`.

NOTA

Las llamadas a semáforos del IPC System V en realidad operan sobre un arreglo, o conjunto, de semáforos, en lugar de hacerlo sobre uno solo de ellos. El deseo de esta explicación, no obstante, es simplificar el tratamiento y presentar el material al lector en lugar de cubrir los semáforos en toda su complejidad. Ya sea que se trabaje con un único semáforo o con muchos al mismo tiempo, el enfoque básico es el mismo, pero resulta importante que el lector comprenda que los semáforos del IPC System V vienen en conjuntos. Personalmente, considero que la interfaz es innecesariamente compleja, y el IPC POSIX estandariza una interfaz más simple pero igualmente potente.

El tratamiento que se efectúa de los semáforos en este capítulo omite también su empleo en otras situaciones en las cuales un proceso tiene muchos hilos de ejecución. Los procesos multi-hilos y el empleo de semáforos en dicho contexto se encuentra más allá del alcance de este libro.

La función `semop` (*abrir semáforo*) constituye el núcleo de las rutinas que involucran semáforos. La misma realiza operaciones sobre uno o más de los semáforos creados o accedidos por la función `semget`. Su prototipo es el siguiente:

`int semop(int semid, struct sembuf *semops, unsigned nops);`

`semid` es un identificador de semáforo previamente retornado por `semget` que vincula el conjunto de semáforos a ser manipulado.

`nops` es el número de elementos presentes en el arreglo de estructuras `sembuf` al cual apunta `semops`.

`sembuf`, a su vez, tiene la siguiente estructura:

```
struct sembuf {
    short sem_num;    /* Numero de semaforo */
    short sem_op;      /* Operacion a llevar a cabo */
    short sem_flg;     /* Indicadores que controlan */
};
```

En las estructuras de patrón `sembuf`, el elemento `sem_num` es un número de semáforo ubicado entre cero y `nsems - 1`, en tanto `sem_op` es la operación a realizar y `sem_flg` modifica con su valor el comportamiento de `semop`'s. El valor de `sem_op` puede ser tanto negativo, cero, o positivo.

Si `sem_op` es positivo, el recurso cuyo acceso es controlado por el semáforo resulta liberado y el valor del respectivo semáforo se incrementa.

Si `sem_op` es negativo, el proceso que efectuó la llamada está indicando que desea aguardar hasta que el acceso al recurso requerido esté despejado, en cuyo momento el semáforo será nuevamente decrementado y el recurso que-

dará bloqueado a fin de que pueda ser utilizado por el proceso que efectuó la llamada.

Si `sem_op` vale cero, finalmente, el proceso que efectuó la llamada se bloqueará (aguardará) hasta que el semáforo pase a valer cero; si ya se encuentra en cero en ese momento, la llamada retorna inmediatamente.

`sem_flg` puede ser `IPC_NOWAIT` (*NO ESPERAR*), que exhibe el comportamiento ya descrito anteriormente (ver “Escritura de un mensaje a una cola”), o `SEM_UNDO` (*DESHACER*), que significa que la operación realizada deberá ser revertida a su estado original cuando el proceso que llamó a `semop` termine.



Ejemplo

El programa que sigue, `crear_semaforo`, crea un semáforo y luego incrementa su valor, haciendo que el recurso imaginario cuyo acceso será controlado por el semáforo recién creado se encuentre desbloqueado o disponible:

```
/* Nombre del programa en Internet: mksem.c */
/*
 * crear_semaforo.c - Crea y decrementa un semaforo
 */
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/sem.h>
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int identificador_semaforo;
    int total_semaforos = 1;          /* Cuantos semaforos crear */
    int indicadores = 0666;           /* Derechos de lectura y modificacion para
    todos los usuarios */
    struct sembuf buf;

    /* Crear el semaforo con derechos de lectura/modificacion para todos los
    usuarios */
    identificador_semaforo = semget(IPC_PRIVATE, total_semaforos, indicadores);
    if(identificador_semaforo < 0) {
        perror("semget");
        exit(EXIT_FAILURE);
    }
    printf("Semaforo creado: %d\n", semid);

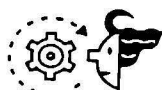
    /* Asignar valores a la estructura de patron sembuf para semop */
    buf.numero_semaforos = 0;          /* Un solo semaforo */
    buf.operacion_a_realizar = 1;       /* Incrementar el semaforo,
    permitir acceso */
    buf.config_indicadores = IPC_NOWAIT; /* Bloquear si se llega a maximo
    valor permitido */
    if((semop(identificador_semaforo, &buf, total_semaforos)) < 0) {
        perror("semop");
    }
}
```

```

        exit(EXIT_FAILURE);
    }
    system("ipcs -s");
    exit(EXIT_SUCCESS);
}

```

La siguiente es la salida de una corrida de `crear_semaforo`. Los valores de los identificadores que se ven posiblemente resulten diferentes en su sistema.



SALIDA

Semaforo creado: 512

--- Semaphore Arrays ---

key	semid	owner	perms	nsems	status
0x00000000	512	kurt_wall	666	1	

El ejemplo utiliza `IPC_PRIVATE` para asegurarse de que el semáforo sea creado tal como se lo requiere, y luego exhibe el valor retornado por `sem_get`, o sea el identificador de ese semáforo. La llamada a `semop` inicializa adecuadamente el semáforo: Como sólo se crea un semáforo, `sem_num` es igual a cero. Como el recurso imaginario no se encuentra en uso (en verdad, este semáforo no ha sido vinculado por el programa con ningún recurso específico), `crear_semaforo` inicializa su valor a 1, el equivalente a desbloqueo. Al no ser requerida una conducta de bloqueo, el indicador `sem_flg` del semáforo se establece a `IPC_NOWAIT`, para que la llamada retorne de forma inmediata. Finalmente, el programa utiliza la llamada a `system` para invocar la utilidad de línea de comandos `ipcs` a fin de confirmar una segunda vez que la estructura IPC requerida, de hecho existe.

Control y remoción de semáforos

El lector ya ha visto funcionar a `msgctl` y `shmctl`, las rutinas que manipulan las colas de mensajes y los segmentos de memoria compartida. Tal como era dable de esperar, la función equivalente para el caso de los semáforos es `semctl`, cuyo prototipo es el siguiente:

✓ `int semctl(int semid, int semnum, int cmd, union semun arg);` *semid, semnum, cmd, semun*

`semid` identifica el conjunto de semáforos que se desea manipular.

`semnum` especifica el semáforo específico en que uno se encuentra interesado. Este libro no toma en cuenta las situaciones en las que hay varios semáforos integrando un conjunto, de modo que `semnum` (en realidad un índice de un arreglo de semáforos) será siempre cero.

El argumento `cmd` (acción) puede ser uno de los valores de la lista siguiente:

- `GETVAL`: Retorna el estado corriente del semáforo (bloqueado o desbloqueado).
- `SETVAL`: Establece el estado corriente del semáforo a `arg.val` (el argumento `semun` se analizará enseguida).
- `GETPID`: Retorna el PID del último proceso que llamó a `semop`.

- **GETNCNT:** Hace que el valor retornado por `semctl` sea el número de procesos aguardando que el semáforo se incremente; es decir, el número de procesos a la espera de luz verde.
- **GETZCNT:** Hace que el valor retornado por `semctl` sea el número de procesos que están aguardando para que el valor del semáforo sea cero.
- **GETALL:** Retorna los valores corrientes de todos los semáforos presentes en el conjunto asociado con `semid`.
- **SETALL:** Asigna a todos los semáforos del conjunto asociado con `semid` los respectivos valores almacenados en `arg.array`.
- **IPC_RMID:** Elimina el semáforo cuyo identificador es `semid`.
- **IPC_SET:** Establece el modo (bits de permiso) en el semáforo.
- **IPC_STAT:** Cada semáforo tiene una estructura de datos, `semid_ds`, que describe enteramente su configuración y comportamiento. `IPC_STAT` copia esta información de configuración al miembro `arg.buf` de la estructura `semun`.

Si la rutina `semctl` fracasa, retorna `-1` y asigna el valor adecuado a la variable `errno`. Si en cambio tiene éxito, retorna un valor entero que puede ser `GETNCNT`, `GETPID`, `GETVAL` o `GETZCNT`, según cuál haya sido el valor de `cmd` que le haya sido transferido.

Tal como el lector debe de haber inferido, el argumento `semun` desempeña un papel vital en la rutina `semctl`. Uno debe definirlo en su código fuente de acuerdo con los lineamientos de la siguiente plantilla:

```
union semun {
    int val;                                /* Valor para SETVAL */
    struct semid_ds *buf;                  /* Buffer de IPC_STAT */
    unsigned short int *array;             /* Buffer de GETALL y SETALL */
};
```



EJEMPLO

Ejemplo

A esta altura el lector debería estar en condiciones de comprender la razón de mis quejas respecto de que la interfaz de semáforos del IPC System V es demasiado complicada para los simples mortales. A pesar de esa dificultad el próximo ejemplo, `eliminar_semaforo`, utiliza la rutina `semctl` para eliminar un semáforo del sistema. Antes de hacerlo se requerirá el empleo de `crear_semaforo` para crear precisamente ese semáforo y luego emplear el identificador del mismo como argumento para la línea de comandos de `eliminar_semaforo`.

```
/* Nombre del programa en Internet: sctl.c */
/*
 * eliminar_semaforo.c - Manipular y eliminar un semaforo
 * Sintaxis: eliminar_semaforo identificador de semaforo
 */
```

```

#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/sem.h>
#include <stdio.h>
#include <stdlib.h>

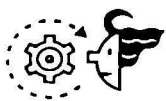
int main(int argc, char *argv[])
{
    int identificador_semaforo;

    if(argc != 2) {
        puts("MOD0 DE EMPLE0: eliminar <identificador de semaforo>");
        exit(EXIT_FAILURE);
    }
    identificador_semaforo = atoi(argv[1]);

    /* Eliminar el semaforo */
    if((semctl(identificador_semaforo, 0, IPC_RMID)) < 0) {
        perror("semctl IPC_RMID");
        exit(EXIT_FAILURE);
    } else {
        puts("Semaforo eliminado");
        system("ipcs -s");
    }

    exit(EXIT_SUCCESS);
}

```

**SALIDA**

La salida que produjo eliminar_semaforo en mi sistema fue la siguiente:

```
$ ./crear_semaforo
```

```
Semaforo creado: 640
```

```
--- Semaphore Arrays ---
```

key	semid	owner	perms	nsems	status
0x00000000	640	kurt_wall	666	1	

```
$ ./eliminar_semaforo 640
```

```
Semaforo eliminado
```

```
--- Semaphore Arrays ---
```

key	semid	owner	perms	nsems	status
-----	-------	-------	-------	-------	--------

El código de `eliminar_semaforo` sencillamente trata de eliminar un semáforo cuyo identificador le es pasado en la línea de comandos. También utiliza una llamada a `system` para ejecutar `ipcs -s` y confirmar así que el semáforo ha sido efectivamente eliminado.

Lo que viene

Este capítulo ha completado nuestra introducción al IPC System V con un análisis de las colas de mensajes y los semáforos. En el próximo capítulo, “Programación de TCP/IP y sockets”, el lector aprenderá los fundamentos de la programación para redes. Los protocolos TCP/IP y de sockets son los más conocidos y más ampliamente utilizados para realizar un IPC entre servidores (*hosts*) diferentes. Luego de que concluya el próximo capítulo, el lector contará con la suficiente información en su poder como para tomar una decisión fundamentada sobre cuál de los diversos mecanismos de IPC se acomoda mejor a sus necesidades.